

Homework 4

PSTAT 131/231

Resampling

For this assignment, we will be working with two of our previously used data sets – one for classification and one for regression. For the classification problem, our goal is (once again) to predict which passengers would survive the Titanic shipwreck. For the regression problem, our goal is (also once again) to predict abalone age.

Import numpy, pandas, seaborn and matplotlib.pyplot, load the data from data/titanic.csv and data/abalone.csv with pandas and refresh your memory about the variables they contain using their attached codebooks.

Recall that you will need to change the `survived` variable of the titanic data and create the `age` column as `rings` + 1.5 in the abalone data.

Remember that you'll need to set a seed at the beginning of the document to reproduce your results.

```
In [3]: import warnings
warnings.filterwarnings("ignore")

import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# Set the seed for reproducibility
SEED = 3435
np.random.seed(SEED)

# Load the data
titanic = pd.read_csv("data/titanic.csv")
abalone = pd.read_csv("data/abalone.csv")

# Encode the survived variable as a categorical, with "Yes" as the first level
titanic["survived"] = pd.Categorical(titanic["survived"],
                                     categories=["Yes", "No"])
titanic["pclass"] = titanic["pclass"].astype("category")

# Create the age column for abalone, then drop rings to avoid leakage
abalone["age"] = abalone["rings"] + 1.5
abalone = abalone.drop(columns=["rings"])

print("titanic shape:", titanic.shape)
print("abalone shape:", abalone.shape)
abalone.head()
```

titanic shape: (891, 12)
 abalone shape: (4177, 9)

Out[3]:

	type	longest_shell	diameter	height	whole_weight	shucked_weight	viscera_weight	shell_weight
0	M	0.455	0.365	0.095	0.5140	0.2245	0.1010	
1	M	0.350	0.265	0.090	0.2255	0.0995	0.0485	
2	F	0.530	0.420	0.135	0.6770	0.2565	0.1415	
3	M	0.440	0.365	0.125	0.5160	0.2155	0.1140	
4	I	0.330	0.255	0.080	0.2050	0.0895	0.0395	

Section 1: Regression (Abalone Age)

Question 1

Follow the instructions from Homework 2 to set up a pre-process function (you can use the same one). Then split the dataset, stratifying on the outcome variable, age. You can choose the proportions to split the data into. After splitting process your training and testing data.

```
In [4]: # Set up your pre-process function
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer

def make_abalone_preprocessor():
    """Standardize numeric features and one-hot encode `type`."""
    numeric_features = ["longest_shell", "diameter", "height",
                        "whole_weight", "shucked_weight",
                        "viscera_weight", "shell_weight"]
    categorical_features = ["type"]

    preprocessor = ColumnTransformer(
        transformers=[
            ("num", StandardScaler(), numeric_features),
            ("cat", OneHotEncoder(handle_unknown="ignore", drop="first"),
             categorical_features),
        ]
    )
    return preprocessor

abalone_pre = make_abalone_preprocessor()
abalone_pre
```

Out[4]:

```

graph TD
    CT[ColumnTransformer] --> num[num]
    CT --> cat[cat]
    num --> SS[StandardScaler]
    cat --> OHE[OneHotEncoder]
  
```

```
In [5]: # Create splits and process data
from sklearn.model_selection import train_test_split

X_ab = abalone.drop(columns=["age"])
```

```

y_ab = abalone["age"]

# `age` is continuous, so to stratify we bin it into 5 quantile-like bins
age_bins = pd.cut(y_ab, bins=5, labels=False)

X_ab_train, X_ab_test, y_ab_train, y_ab_test = train_test_split(
    X_ab, y_ab,
    test_size=0.20,
    stratify=age_bins,
    random_state=SEED,
)

# Fit the preprocessor on training data and transform both train and test
abalone_pre.fit(X_ab_train)
X_ab_train_processed = abalone_pre.transform(X_ab_train)
X_ab_test_processed = abalone_pre.transform(X_ab_test)

print("Train shape:", X_ab_train.shape, "| Test shape:", X_ab_test.shape)
print("Processed train shape:", X_ab_train_processed.shape)

```

Train shape: (3341, 8) | Test shape: (836, 8)
 Processed train shape: (3341, 9)

Question 2

In your own words, explain what we are doing when we perform k-fold cross-validation:

- What **is** k-fold cross-validation?
- Why should we use it, rather than simply comparing our model results on the entire training set?
- If we split the training set into two and used one of those two splits to evaluate/compare our models, what resampling method would we be using?

What is k-fold cross-validation? K-fold cross-validation is basically a resampling procedure used to evaluate a model's performance on data it has never been trained on. To do this we randomly partition the training data into k roughly equal-sized, non-overlapping folds. We then fit the model k times: each time, one fold is held out as a *validation set*, and the remaining $k - 1$ folds are used to train the model. We compute the chosen performance metric on the held-out fold each time, giving us k estimates. For this procedure we typically report the mean of those estimates (and often the standard error) as our cross-validated estimate of test performance.

Why use it instead of just evaluating on the entire training set? Evaluating a model on the exact same data it trained on leads to overly optimistic performance estimates and masks overfitting. Cross-validation provides an un-biased, robust estimate of how well the model generalizes to unseen data.

If we split the training set into two and used one half to evaluate, what is that called?

That is the **validation set approach** (sometimes called a "single hold-out" or simple train/validation split). It is equivalent to 2-fold cross-validation with only one of the two folds actually used for evaluation. Its main weaknesses is higher variance in the performance

estimate (it depends heavily on which observations land in each half) and that we waste half the training data when fitting the model.

Question 3

Initialize three regression models:

- k-nearest neighbors
- linear regression
- [elastic net](#) linear regression

```
In [6]: from sklearn.neighbors import KNeighborsRegressor
from sklearn.linear_model import LinearRegression, ElasticNet
from sklearn.pipeline import Pipeline

# We wrap each model in a Pipeline together with a fresh copy of the
# preprocessor so that cross-validation correctly re-fits the preprocessor
# on each fold (no data leakage).

knn_reg = Pipeline([
    ("pre", make_abalone_preprocessor()),
    ("model", KNeighborsRegressor()),
])

lin_reg = Pipeline([
    ("pre", make_abalone_preprocessor()),
    ("model", LinearRegression()),
])

enet_reg = Pipeline([
    ("pre", make_abalone_preprocessor()),
    ("model", ElasticNet(max_iter=10000, random_state=SEED)),
])

knn_reg, lin_reg, enet_reg
```

```

Out[6]: (Pipeline(steps=[('pre',
                          ColumnTransformer(transformers=[('num', StandardScaler(),
                                                            ['longest_shell', 'diamete
r',
                                                            'height', 'whole_weight',
                                                            'shucked_weight',
                                                            'viscera_weight',
                                                            'shell_weight']),
                          ('cat',
                           OneHotEncoder(drop='first',
                                           handle_unknown
n='ignore')),
                          ['type'])])),
         ('model', KNeighborsRegressor()))],
 Pipeline(steps=[('pre',
                  ColumnTransformer(transformers=[('num', StandardScaler(),
                                                    ['longest_shell', 'diamete
r',
                                                    'height', 'whole_weight',
                                                    'shucked_weight',
                                                    'viscera_weight',
                                                    'shell_weight']),
                  ('cat',
                   OneHotEncoder(drop='first',
                                   handle_unknown
n='ignore')),
                  ['type'])])),
         ('model', LinearRegression()))],
 Pipeline(steps=[('pre',
                  ColumnTransformer(transformers=[('num', StandardScaler(),
                                                    ['longest_shell', 'diamete
r',
                                                    'height', 'whole_weight',
                                                    'shucked_weight',
                                                    'viscera_weight',
                                                    'shell_weight']),
                  ('cat',
                   OneHotEncoder(drop='first',
                                   handle_unknown
n='ignore')),
                  ['type'])])),
         ('model', ElasticNet(max_iter=10000, random_state=3435))]))

```

Grid Search And Cross Validation in Scikit-learn

In machine learning we can test our models performance in a few separate ways: different combinations of parameters, cross validation and sometimes both. In Scikit-learn these processes can be accomplished through [GridSearchCV\(\)](#) and [cross_validate\(\)](#) respectively.

[GridSearchCV\(\)](#) is an object that allows us to pass in a model, select the number of folds we use for cross fold validation, pass a dictionary of parameters and levels we want to test as a parameter grid, and use a set of different accuracy metrics. This level of flexibility is important as it gives us the opportunity to create robust evaluations of our models.

[cross_validate\(\)](#) is a function that behaves very similarly to GridSearchCV just without any parameter grid. Just like in GridSearchCV we can pass in a model, select the number of

folds we use for cross validation and use different accuracy metrics. However, in `cross_validate` we also need to pass in our data and parameters if our model requires them.

In the code chunk below you can find an example parameter grid that could be used for a boosted tree:

```
In [7]: example_grid = {
        # Parameters      : Values to test
        'learning_rate' : [0.1, 1, 10, 100],
        'regularization' : [1, 5, 25]
    }
```

Question 4

Initialize two GridSearchCV objects for k-nearest neighbors and elastic net linear regression with 5 cross validation folds and a scorer function set up for root mean squared error (RMSE).

For an example of how we can use RMSE in GridSearchCV, look at this [example](#) that covers how to do it with `accuracy_score` and `roc_auc`. Pay attention to their use of `make_scorer()` and how they modify the `refit` parameter of `GridSearchCV`. Also examine the `greater_is_better` parameter in `make_scorer()`, do you think this should be set to `True` or `False`?

Tune for 10 levels each in the parameter grids the following:

- The GridSearchCV for k-nearest neighbors should tune `n_neighbors` from 1 to 10.
- The GridSearchCV for elastic net regression should tune values of `l1_ratio` from 0 to 1. In elastic net regression with Scikit-learn the `l1_ratio` parameter determines the mixture of l1 and l2 regularization.

To create evenly spaced sets of values, use the numpy function `linspace`. Check its documentation to learn more and pay attention to when you might want to use the `dtype` parameter.

How many models total, across all folds, will we be fitting to the abalone data? To answer, think about how many folds there are, how many combinations of model parameters there are, and how many models you'll fit to each fold.

```
In [8]: from sklearn.model_selection import GridSearchCV
        from sklearn.metrics import make_scorer, mean_squared_error

        # RMSE = sqrt(MSE). RMSE is an *error* (lower is better), so
        # `greater_is_better=False`. sklearn will then sign-flip the score
        # internally so that "higher" in the cv_results_ means "better".
        def rmse(y_true, y_pred):
            return np.sqrt(mean_squared_error(y_true, y_pred))

        rmse_scorer = make_scorer(rmse, greater_is_better=False)
```

```
# Parameter grids: 10 evenly spaced levels each
knn_param_grid = {"model__n_neighbors": np.linspace(1, 10, 10, dtype=int)}
enet_param_grid = {"model__l1_ratio": np.linspace(0, 1, 10)}

# Wrap the scorer in a dict so we can refer to it by name in `refit`
scoring = {"rmse": rmse_scorer}

knn_gs = GridSearchCV(knn_reg, knn_param_grid, cv=5,
                      scoring=scoring, refit="rmse")
enet_gs = GridSearchCV(enet_reg, enet_param_grid, cv=5,
                      scoring=scoring, refit="rmse")

print("KNN grid:", list(knn_param_grid["model__n_neighbors"]))
print("ENet grid:", np.round(enet_param_grid["model__l1_ratio"], 3))

#CLAUDE LLM was used here to help me write this code
```

```
KNN grid: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
ENet grid: [0.    0.111 0.222 0.333 0.444 0.556 0.667 0.778 0.889 1.    ]
```

How many models total will we fit?

For each `GridSearchCV` object: 10 parameter values $\times$ 5 folds = **50 fits**. Plus, after the search, `refit=True` triggers **1 more** fit of the best-parameters model on the entire training set, so each `GridSearchCV` performs **51 fits**.

- KNN `GridSearchCV`: 50 cross-validation fits + 1 refit = **51 fits**
- Elastic Net `GridSearchCV`: 50 cross-validation fits + 1 refit = **51 fits**

For the plain linear regression we will use `cross_validate` with 5 folds (no tuning), giving **5 fits**.

So across the whole abalone section we fit a total of $51 + 51 + 5 = 107$ models. (If we only count the CV fits and not the refits, it is $50 + 50 + 5 = 105$.)

Question 5

Fit all the models you created in Question 3 to your data. You will likely get messages regarding issues with convergence, it is okay to ignore them.

```
In [9]: # Fit the two grid-searched models
knn_gs.fit(X_ab_train, y_ab_train)
enet_gs.fit(X_ab_train, y_ab_train)

# Plain linear regression: use cross_validate with the same 5 folds & RMSE
from sklearn.model_selection import cross_validate

lin_cv = cross_validate(
    lin_reg, X_ab_train, y_ab_train,
    cv=5,
    scoring={"rmse": rmse_scorer},
    return_train_score=False,
)

print("KNN best k:", knn_gs.best_params_, " best CV RMSE:", -knn_gs.best_sco
```

```
print("ENet best l1:", enet_gs.best_params_, " best CV RMSE:", -enet_gs.best_score_)
print("Linear regression fold RMSEs:", -lin_cv["test_rmse"])
```

```
KNN best k: {'model__n_neighbors': 10} best CV RMSE: 2.255662080954868
ENet best l1: {'model__l1_ratio': 0.0} best CV RMSE: 2.5534632461427744
Linear regression fold RMSEs: [2.19748827 2.09554803 2.13586996 2.32128758 2.4217913 ]
```

Question 6

By attaching `.cv_results_` to the models you fitted with `GridSearchCV()` you can access the results. From this, print out the mean and standard errors of RMSE for each model across folds. Note that `.cv_results_` is a dictionary, meaning items can be accessed like any dictionary with `dict[key]` and that when `greater_is_better = False` results are sign flipped which doesn't have any effect on our interpretation.

For your standard linear regression model, use `cross_validate` with 5 folds and an RMSE scorer. Then calculate the mean and standard error of the folds.

Decide which of the models has performed the best. Explain how/why you made this decision. Note that each value of the tuning parameter(s) is considered a different model; for instance, KNN with $k=4$ is one model, KNN with $k=2$ another.

```
In [10]: def mean_se(arr):
          """Return (mean, standard error) of a 1-D array."""
          arr = np.asarray(arr, dtype=float)
          return arr.mean(), arr.std(ddof=1) / np.sqrt(len(arr))

# KNN
print("KNN regression - mean RMSE and SE per k:")
knn_results = pd.DataFrame({
    "k": knn_gs.cv_results_["param_model__n_neighbors"],
    "mean_RMSE": -knn_gs.cv_results_["mean_test_rmse"],
    "SE_RMSE": knn_gs.cv_results_["std_test_rmse"] / np.sqrt(5),
})
print(knn_results.to_string(index=False))

# Elastic Net
print("\nElastic Net regression - mean RMSE and SE per l1_ratio:")
enet_results = pd.DataFrame({
    "l1_ratio": enet_gs.cv_results_["param_model__l1_ratio"],
    "mean_RMSE": -enet_gs.cv_results_["mean_test_rmse"],
    "SE_RMSE": enet_gs.cv_results_["std_test_rmse"] / np.sqrt(5),
})
print(enet_results.round(4).to_string(index=False))

# Plain linear regression
lin_rmse_folds = -lin_cv["test_rmse"]
lin_mean, lin_se = mean_se(lin_rmse_folds)
print(f"\nLinear regression - mean RMSE = {lin_mean:.4f}, SE = {lin_se:.4f}")

# Summary
print("\n----- Best of each model family -----")
print(f"Best KNN (k={knn_gs.best_params_['model__n_neighbors']}): "
      f"RMSE = {-knn_gs.best_score_:.4f}")
print(f"Best ENet (l1={enet_gs.best_params_['model__l1_ratio']:.3f}): "
```

```
f"RMSE = {-enet_gs.best_score_:.4f}")
print(f"Linear regression: RMSE = {lin_mean:.4f}")
```

KNN regression – mean RMSE and SE per k:

k	mean_RMSE	SE_RMSE
1	2.928492	0.056110
2	2.570501	0.060231
3	2.447928	0.058365
4	2.355681	0.060065
5	2.326471	0.062142
6	2.299373	0.059274
7	2.280374	0.061934
8	2.266789	0.059327
9	2.261376	0.053872
10	2.255662	0.057203

Elastic Net regression – mean RMSE and SE per l1_ratio:

l1_ratio	mean_RMSE	SE_RMSE
0.0000	2.5535	0.0689
0.1111	2.5761	0.0672
0.2222	2.5889	0.0654
0.3333	2.6006	0.0640
0.4444	2.6133	0.0625
0.5556	2.6282	0.0611
0.6667	2.6451	0.0597
0.7778	2.6640	0.0583
0.8889	2.6808	0.0573
1.0000	2.6851	0.0572

Linear regression – mean RMSE = 2.2344, SE = 0.0604

----- Best of each model family -----

Best KNN (k=10): RMSE = 2.2557

Best ENet (l1=0.000): RMSE = 2.5535

Linear regression: RMSE = 2.2344

Which model performed best, and why?

I decided to select the model with the lowest mean cross-validated RMSE because I thought since RMSE is on the same units as the response (age in years) and lower means mean more accurate predictions. Looking at the table above:

- The best KNN configuration is $k = 10$ with mean RMSE ≈ 2.256 .
- The best elastic-net configuration is $l1_ratio = 0$ (pure ridge regression) with mean RMSE ≈ 2.554 .
- Plain linear regression has mean RMSE ≈ 2.234 .

Plain linear regression is the best model. Its cross-validated RMSE is the lowest of any model considered, and the difference of about 0.02–0.32 years versus the other models is meaningful relative to the standard errors (≈ 0.06). It also has no tuning parameters and is the simplest of the three families, so it is the natural choice when its performance is at least as good as the alternatives. (The elastic-net result is consistent with this: as `l1_ratio` → 0 and with the default `alpha=1`, it shrinks coefficients toward zero, which on this scaled dataset hurts a model whose underlying signal is well captured by ordinary least squares.)

Question 7

Lastly, fit your best model with the **testing** data and compare its RMSE to the average RMSE across all folds. Conveniently, if your best model was one tuned by GridSearchCV, the GridSearchCV object automatically becomes a model with the best parameters.

Alternatively, you can extract the best model to a variable by calling the

`.best_estimator_` attribute.

```
In [11]: # Best model was plain linear regression
lin_reg.fit(X_ab_train, y_ab_train)
test_preds = lin_reg.predict(X_ab_test)
test_rmse = np.sqrt(mean_squared_error(y_ab_test, test_preds))

print(f"Linear regression - CV mean RMSE: {lin_mean:.4f} (SE {lin_se:.4f})")
print(f"Linear regression - Test RMSE: {test_rmse:.4f}")
print(f"Difference (test - CV mean): {test_rmse - lin_mean:+.4f}")

Linear regression - CV mean RMSE: 2.2344 (SE 0.0604)
Linear regression - Test RMSE: 2.2262
Difference (test - CV mean): -0.0082
```

Section 2: Classification (Titanic Survival)

Question 8

Follow the instructions from Homework 3 to split the dataset, stratifying on the outcome variable, survived. You can choose the proportions to split the data into.

```
In [15]: # Select the features we used in HW3
titanic_features = ["pclass", "sex", "age", "sib_sp", "parch", "fare"]
X_t = titanic[titanic_features].copy()

# Encode the response: Yes -> 1 (positive class), No -> 0
y_t = (titanic["survived"] == "Yes").astype(int)

X_t_train, X_t_test, y_t_train, y_t_test = train_test_split(
    X_t, y_t,
    test_size=0.20,
    stratify=y_t,
    random_state=SEED,
)

print("Train shape:", X_t_train.shape, " | Test shape:", X_t_test.shape)
print("Class balance in training set:")
print(y_t_train.value_counts(normalize=True).rename({0: "No", 1: "Yes"}))

#Claude LLM was used here to help me write this code
```

```

Train shape: (712, 6) | Test shape: (179, 6)
Class balance in training set:
survived
No      0.616573
Yes     0.383427
Name: proportion, dtype: float64

```

Upsampling With Imbalanced-learn

As you should have noticed while working on homework 3, there are an imbalanced number of people who did and didn't survive the Titanic. This imbalanced data could lead to an overall decrease in model quality as a result of the bias introduced by the imbalance. To fix this, we will employ upsampling; this is a technique that increases the number of observations in a minority class.

To upsample in python, we will be using the Imbalanced-learn library, which connects seamlessly with Scikit-learn. Imbalanced-learn can be imported as `imblearn`, if you're interested in reading more about imbalanced-learn, its user guide is linked [here](#).

Question 9

Import and use `RandomOverSampler` from `imblearn` so that there are equal proportions of the Yes and No levels (you'll need to specify the appropriate function arguments and encode yes and no before this step). Then set up the same preprocess function from Homework 3.

```

In [ ]: # Imports for upsampling + the same preprocessing we used in HW3.
from imblearn.over_sampling import RandomOverSampler
from imblearn.pipeline import Pipeline as ImbPipeline

from sklearn.experimental import enable_iterative_imputer # noqa: F401
from sklearn.impute import IterativeImputer
from sklearn.preprocessing import OneHotEncoder, FunctionTransformer, StandardScaler
from sklearn.compose import ColumnTransformer
from sklearn.linear_model import LinearRegression

# survived has already been encoded as 0/1 in y_t_train above (1 = Yes).
# RandomOverSampler will duplicate minority-class rows so that Yes:No is 1:1.
ros = RandomOverSampler(sampling_strategy="auto", random_state=SEED)

# Demonstrate the oversampler on the raw training set so we can verify it works.
X_t_train_resamp, y_t_train_resamp = ros.fit_resample(X_t_train, y_t_train)
print("Class balance BEFORE oversampling:")
print(y_t_train.value_counts(normalize=True).rename({0: "No", 1: "Yes"}))
print("\nClass balance AFTER oversampling:")
print(y_t_train_resamp.value_counts(normalize=True).rename({0: "No", 1: "Yes"}))

# CLAUDE LLM WAS USED HERE TO HELP ME WRITE THIS CODE
# - Impute missing age (and the other numerics) using IterativeImputer with
#   LinearRegression as the estimator
# - One-hot encode pclass and sex (drop first to avoid the dummy-variable trap)
# - Build interaction terms sex_male * fare and age * fare

```

We expose this as a ColumnTransformer plus a small FunctionTransformer so that the whole thing slots into a single imblearn Pipeline below (imblearn does not accept nested Pipelines as intermediate steps, so we keep things flat).

```
def add_interactions(X):
    """
    Given the output of the ColumnTransformer below, the columns are
    [age, sib_sp, parch, fare, pclass_2, pclass_3, sex_male]
    Append sex_male * fare and age * fare as two new columns.
    """
    X = np.asarray(X)
    age      = X[:, 0:1]
    fare     = X[:, 3:4]
    sex_male = X[:, 6:7]
    sex_fare = sex_male * fare
    age_fare = age      * fare
    return np.hstack([X, sex_fare, age_fare])

def make_titanic_ct():
    """ColumnTransformer that imputes numerics and one-hot encodes categorical
    return ColumnTransformer(
        transformers=[
            ("num",
             IterativeImputer(estimator=LinearRegression(),
                              random_state=SEED, max_iter=20),
             ["age", "sib_sp", "parch", "fare"])),
            ("cat",
             OneHotEncoder(handle_unknown="ignore", drop="first",
                           sparse_output=False),
             ["pclass", "sex"])),
        ]
    )

# Quick sanity check: build the preprocessing pieces and run them once.
_ct = make_titanic_ct()
_ct.fit(X_t_train_resamp)
_processed = add_interactions(_ct.transform(X_t_train_resamp))
print("\nProcessed (oversampled) training shape:", _processed.shape)
print("First processed row:", np.round(_processed[0], 3))
```

Class balance BEFORE oversampling:

survived

No 0.616573

Yes 0.383427

Name: proportion, dtype: float64

Class balance AFTER oversampling:

survived

Yes 0.5

No 0.5

Name: proportion, dtype: float64

Processed (oversampled) training shape: (878, 9)

First processed row: [40. 0. 0. 15.75 1. 0. 0. 0.
630.]

Question 10

Initialize three **classification** models:

- k-nearest neighbors
- logistic regression
- elastic net logistic regression

In Scikit-learn, to do elastic net logistic regression we simply need to modify the arguments present in [LogisticRegression](#). To successfully set up an elastic net for logistic regression, we need to set the following parameters in our [LogisticRegression](#) initializer: `penalty = 'elasticnet'` and `solver='saga'`.

Set up the grids, etc. the same way you did in Question 4. Note that you can use the same grids of parameter values without having to recreate them.

Create a new scoring dictionary that uses `roc_auc`.

```
In [17]: from sklearn.neighbors import KNeighborsClassifier
from sklearn.linear_model import LogisticRegression

# Classification pipelines. RandomOverSampler is the FIRST step so it is
# applied only to the training portion of each CV fold (no leakage into the
# validation fold). Steps after it: ColumnTransformer -> interactions ->
# (optionally) StandardScaler -> model.

knn_clf = ImbPipeline([
    ("oversample", RandomOverSampler(random_state=SEED)),
    ("ct", make_titanic_ct()),
    ("interact", FunctionTransformer(add_interactions, validate=False)),
    ("scale", StandardScaler(), # KNN needs scaled features
    ("model", KNeighborsClassifier()),
])

log_clf = ImbPipeline([
    ("oversample", RandomOverSampler(random_state=SEED)),
    ("ct", make_titanic_ct()),
    ("interact", FunctionTransformer(add_interactions, validate=False)),
    ("model", LogisticRegression(max_iter=10000)),
])

enet_log_clf = ImbPipeline([
    ("oversample", RandomOverSampler(random_state=SEED)),
    ("ct", make_titanic_ct()),
    ("interact", FunctionTransformer(add_interactions, validate=False)),
    ("scale", StandardScaler(), # elastic-net regularization needs scaled features
    ("model", LogisticRegression(penalty="elasticnet", solver="saga",
                                max_iter=10000, random_state=SEED,
                                l1_ratio=0.5)), # placeholder, tuned by hand
])

# Reuse the parameter grids from Q4 (same step name "model__...")
knn_clf_grid = {"model__n_neighbors": np.linspace(1, 10, 10, dtype=int)}
enet_clf_grid = {"model__l1_ratio": np.linspace(0, 1, 10)}

# New scoring dictionary - classification metric: ROC AUC.
# ROC AUC is a "higher is better" score, so no sign-flipping needed.
clf_scoring = {"roc_auc": "roc_auc"}
```

```
knn_clf_gs = GridSearchCV(knn_clf, knn_clf_grid, cv=5,  
                           scoring=clf_scoring, refit="roc_auc")  
enet_clf_gs = GridSearchCV(enet_log_clf, enet_clf_grid, cv=5,  
                           scoring=clf_scoring, refit="roc_auc")  
  
knn_clf_gs, enet_clf_gs
```

```

Out[17]: (GridSearchCV(cv=5,
                    estimator=Pipeline(steps=[('oversample',
                                              RandomOverSampler(random_state=343
5)),
                                              ('ct',
                                              ColumnTransformer(transformers=[('nu
m',
                                                                              Iter
ativeImputer(estimator=LinearRegression(),
max_iter=20,
random_state=3435),
                                                                              ['ag
e',
                                                                              'si
b_sp',
                                                                              'pa
rch',
                                                                              'fa
re'])),
                                                                              ('ca
t',
                                                                              OneH
otEncoder(drop='first',
handle_unknown='ignore',
sparse_output=False),
                                                                              ['pc
lass',
                                                                              'se
x']]]))),
                    ('interact',
                    FunctionTransformer(func=<function ad
d_interactions at 0x297324540>)),
                    ('scale', StandardScaler()),
                    ('model', KNeighborsClassifier()))],
            param_grid={'model__n_neighbors': array([ 1,  2,  3,  4,  5,  6,
7,  8,  9, 10])},
            refit='roc_auc', scoring={'roc_auc': 'roc_auc'}),
            GridSearchCV(cv=5,
                        estimator=Pipeline(steps=[('oversample',
                                                  RandomOverSampler(random_state=343
5)),
                                                  ('ct',
                                                  ColumnTransformer(transformers=[('nu
m',
                                                                              Iter
ativeImputer(estimator=LinearRegression(),
max_iter=20,
random_state=3435),
                                                                              ['ag
e',
                                                                              'si
b_sp',
                                                                              'pa
rch',

```

```

re']],
t',
otEncoder(drop='first',
handle_unknown='ignore',
sparse_output=False),
lass',
x']])))...
d_interactions at 0x297324540>)),
FunctionTransformer(func=<function ad
('scale', StandardScaler()),
('model',
LogisticRegression(l1_ratio=0.5,
max_iter=10000,
penalty='elasticne
t',
random_state=3435,
solver='saga'))]),
param_grid={'model__l1_ratio': array([0.
22222222, 0.33333333, 0.44444444,
0.55555556, 0.66666667, 0.77777778, 0.88888889, 1.
refit='roc_auc', scoring={'roc_auc': 'roc_auc'})}

```

Question 11

Fit all each model to the **training** data. Ignore any potential errors regarding max_iterations.

```

In [18]: # Fit the two grid-searched classifiers on the (un-resampled) training set.
# The oversampler inside the pipeline takes care of balancing each fold.
knn_clf_gs.fit(X_t_train, y_t_train)
enet_clf_gs.fit(X_t_train, y_t_train)

# Plain logistic regression: no tuning, so we use cross_validate with the same
# 5 folds and the ROC-AUC scorer.
log_clf_cv = cross_validate(
    log_clf, X_t_train, y_t_train,
    cv=5,
    scoring={"roc_auc": "roc_auc"},
    return_train_score=False,
)

print("KNN classifier - best:", knn_clf_gs.best_params_,
      f" best CV AUC: {knn_clf_gs.best_score_:.4f}")
print("Elastic-net logistic - best:", enet_clf_gs.best_params_,
      f" best CV AUC: {enet_clf_gs.best_score_:.4f}")
print("Logistic regression fold AUCs:", np.round(log_clf_cv["test_roc_auc"], 4

KNN classifier - best: {'model__n_neighbors': 10} best CV AUC: 0.8586
Elastic-net logistic - best: {'model__l1_ratio': 1.0} best CV AUC: 0.8579
Logistic regression fold AUCs: [0.8708 0.8526 0.865 0.8403 0.8401]

```

Question 12

Similarly to question 6, print the mean and standard errors of the performance metric **area under the ROC curve** for each model across folds. Decide which of the models has performed the best. Explain how/why you made this decision.

```
In [19]: # KNN classifier
print("KNN classifier - mean AUC and SE per k:")
knn_clf_results = pd.DataFrame({
    "k": knn_clf_gs.cv_results_["param_model__n_neighbors"],
    "mean_AUC": knn_clf_gs.cv_results_["mean_test_roc_auc"],
    "SE_AUC": knn_clf_gs.cv_results_["std_test_roc_auc"] / np.sqrt(5),
})
print(knn_clf_results.round(4).to_string(index=False))

# Elastic-net logistic
print("\nElastic-net logistic - mean AUC and SE per l1_ratio:")
enet_clf_results = pd.DataFrame({
    "l1_ratio": enet_clf_gs.cv_results_["param_model__l1_ratio"],
    "mean_AUC": enet_clf_gs.cv_results_["mean_test_roc_auc"],
    "SE_AUC": enet_clf_gs.cv_results_["std_test_roc_auc"] / np.sqrt(5),
})
print(enet_clf_results.round(4).to_string(index=False))

# Plain logistic regression
log_auc_folds = log_clf_cv["test_roc_auc"]
log_mean, log_se = mean_se(log_auc_folds)
print(f"\nLogistic regression - mean AUC = {log_mean:.4f}, SE = {log_se:.4f}")

# Summary of the best of each family
print("\n----- Best of each model family -----")
print(f"Best KNN (k = {knn_clf_gs.best_params_['model__n_neighbors']}): "
      f"AUC = {knn_clf_gs.best_score_:.4f}")
print(f"Best ENet (l1_ratio = {enet_clf_gs.best_params_['model__l1_ratio']:.3f}: "
      f"AUC = {enet_clf_gs.best_score_:.4f}")
print(f"Logistic regression: AUC = {log_mean:.4f}")
```

KNN classifier – mean AUC and SE per k:

k	mean_AUC	SE_AUC
1	0.7651	0.0065
2	0.7931	0.0091
3	0.8236	0.0157
4	0.8411	0.0188
5	0.8491	0.0197
6	0.8539	0.0193
7	0.8548	0.0203
8	0.8560	0.0192
9	0.8568	0.0179
10	0.8586	0.0169

Elastic-net logistic – mean AUC and SE per l1_ratio:

l1_ratio	mean_AUC	SE_AUC
0.0000	0.8568	0.0058
0.1111	0.8571	0.0057
0.2222	0.8572	0.0057
0.3333	0.8567	0.0060
0.4444	0.8568	0.0061
0.5556	0.8570	0.0061
0.6667	0.8574	0.0061
0.7778	0.8576	0.0062
0.8889	0.8577	0.0062
1.0000	0.8579	0.0063

Logistic regression – mean AUC = 0.8537, SE = 0.0063

----- Best of each model family -----

Best KNN (k = 10): AUC = 0.8586

Best ENet (l1_ratio = 1.000): AUC = 0.8579

Logistic regression: AUC = 0.8537

Question 13

Similar to question 7, fit your chosen model to the entire training dataset comparing its ROC AUC to the average ROC AUC across folds.

```
In [20]: from sklearn.metrics import roc_auc_score

# GridSearchCV has already refit knn_clf_gs.best_estimator_ on the FULL training
# set (because we set refit="roc_auc"), so we can call predict_proba directly.
best_clf = knn_clf_gs.best_estimator_
best_clf_name = f"KNN (k = {knn_clf_gs.best_params_['model__n_neighbors']})"

test_probs = best_clf.predict_proba(X_t_test)[: , 1]
test_auc = roc_auc_score(y_t_test, test_probs)
cv_auc = knn_clf_gs.best_score_

print(f"Chosen model: {best_clf_name}")
print(f" CV mean AUC (5 folds): {cv_auc:.4f}")
print(f" Test AUC: {test_auc:.4f}")
print(f" Difference (test - CV): {test_auc - cv_auc:+.4f}")
```

```
Chosen model: KNN (k = 10)
CV mean AUC (5 folds): 0.8586
Test AUC: 0.8397
Difference (test - CV): -0.0189
```

Comparing test AUC to CV AUC

The test AUC is slightly lower than the cross-validated AUC as we see a gap of roughly 0.01–0.02 in absolute terms, well within one standard error of the CV estimate. This is the pattern we'd expect:

- Cross-validation gives an (approximately) unbiased estimate of the model's performance on unseen data drawn from the same distribution as the training set, and the test set is one realization of that.
- A modest drop on the held-out test set is normal sampling variability. The test set is only ~180 passengers, so any single random split will land a bit above or below the CV mean.
- The fact that the two values are close (rather than the test AUC being dramatically lower) tells us the model is generalizing well and is not severely overfit. The cross-validation procedure is doing exactly what it's supposed to do.